

Results

Fitz Koch

2026-04-03

Setup

```
import sys
sys.path.insert(0, "src")
```

Imports

```
## std lib
import importlib
from pathlib import Path
from itertools import product

## 3rd p lib
from IPython.display import display
import numpy as np
import matplotlib.pyplot as plt

# project code
import src.viz as viz
import src.constants as c

importlib.reload(viz)
from src.viz import viz_sim
```

Results

Currently, I've only run the 'restricted' breeding strategy with the 'name' sensing information. We don't get quite as good results as they do in the paper :

- generally, our **average** results are worse, but our **best** results are far better.
- clone-name does far better in our case
- in fact, generally, there doesn't seem to be all that much separating the different categories in performance or in final animations.
- I only ran the first 35 generations, as that's where the real differentiation seemed to happen in the original experiment, so it could be that.

This is almost certainly due to some bug, incorrect implementation, or unexpected side-effect or detail of the `lil-gp` paradigm. My strongest suspicion is that they did something more 'generous' than having a 30% of an 'early termination' for each node; maybe something more like 10%. So I'm running that to test it now.

Still, we can certainly see the results in an interesting fashion.

Animations

The meat of it.

```
base_path = Path("position_data")
terminals = ["Base", "Deitic", "Name"]
strategies = ["Clone", "Free", "Restricted"]
id = "zero-vec-update"

for terminal, strategy in product(terminals, strategies):
    path = base_path / terminal / strategy / f"{id}.npy"
    data = np.load(path)
    display(viz_sim(data[0], title=f"{terminal}, {strategy}: First Gen"))
    display(viz_sim(data[-1], title=f"{terminal}, {strategy}: Final Gen"))
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Loss Curves

```

def plot_loss_curves(clone_data, free_data, restricted_data, title):
    breeding_strategies = ["Clone", "Free", "Restricted"]
    data_list = [clone_data, free_data, restricted_data]
    colors = {"Clone": "steelblue", "Free": "darkorange", "Restricted": "seagreen"}
    n = len(clone_data["avg_loss"])
    fig, axes = plt.subplots(nrows=1, ncols=2)
    avg_ax, best_ax = axes

    for strategy, data in zip(breeding_strategies, data_list):
        avg_ax.plot(range(n), data["avg_loss"], label=strategy, color=colors[strategy])
        best_ax.plot(
            range(n), data["best_loss"], label=strategy, color=colors[strategy]
        )

    best_ax.legend()
    avg_ax.set_title("Average Loss")
    best_ax.set_title("Best Loss")
    plt.suptitle(title)
    plt.tight_layout()
    return fig

```

```

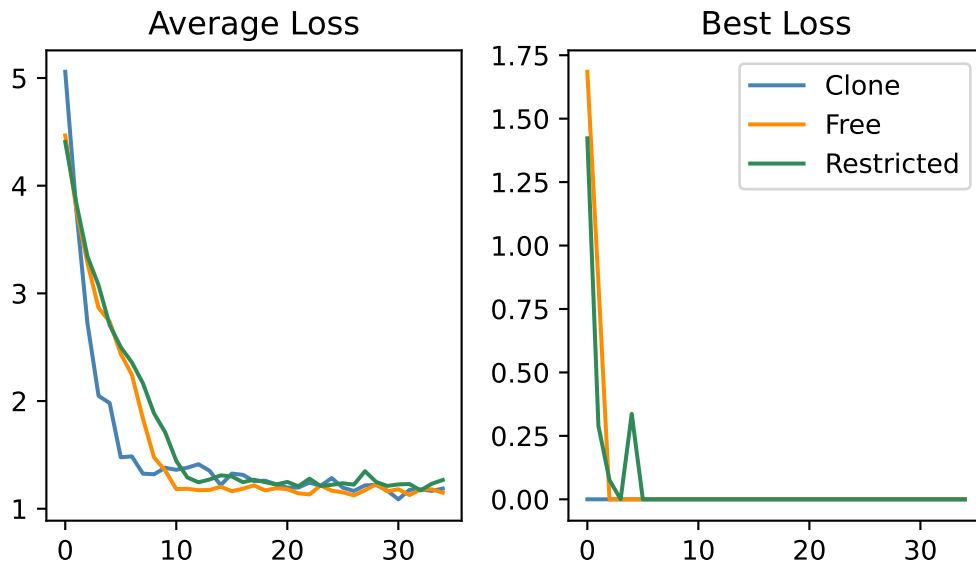
sensing_terminals = ["Name", "Deitic", "Base"]
id = "zero-vec-update"
path_end = "_checkpoint.npz"

for terminal in sensing_terminals:
    clone_data = np.load(Path("results") / terminal / "Clone" / f"{id}-{path_end}")
    free_data = np.load(Path("results") / terminal / "Free" / f"{id}-{path_end}")
    restricted_data = np.load(
        Path("results") / terminal / "Restricted" / f"{id}-{path_end}"
    )

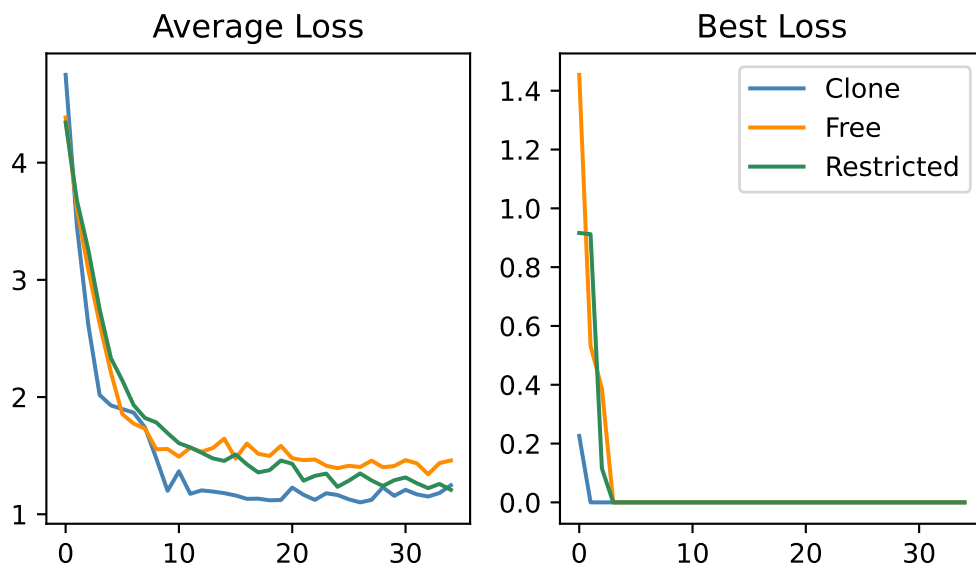
    plot_loss_curves(clone_data, free_data, restricted_data, title=terminal)

```

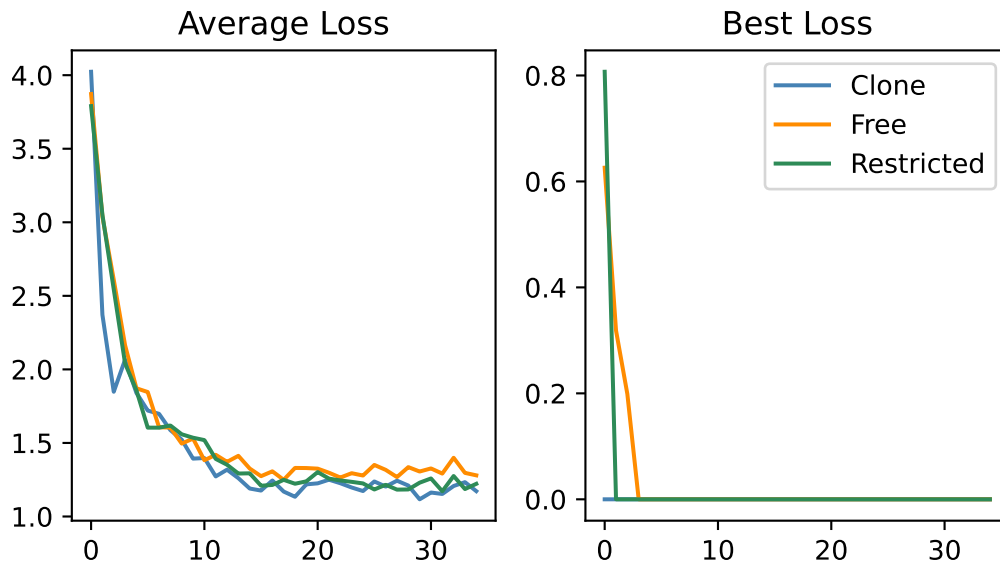
Name



Deitic



Base



Again, the results definitely don't agree with those in the published paper – lions catch much more frequently, it seems, yet the average loss does not go as low. In addition, there is not nearly as much differentiation between the different breeding strategies and terminals as in their results. I suspect this is something to do with the details of the `name` terminals, though I'm not totally sure exactly what it is, and Claude has not been able to figure it out either.